

TRABAJO PRÁCTICO INTEGRADOR 2026

Paradigmas y Lenguajes



Tema: Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Grupo N°8

Integrantes:

Aranda Gabriel Ángel

Barrios Marcos Gabriel

Fernández Mirta Noemi

Gaitán Carla Evangelina

Introducción

La programación funcional es un paradigma que se basa en la construcción de programas mediante funciones que transforman datos de entrada en resultados, siguiendo un enfoque declarativo. En este modelo, el comportamiento del programa se expresa a través de la aplicación de funciones, priorizando la claridad y la modularidad del código.

En este trabajo práctico se implementaron soluciones en Common Lisp cumpliendo con los requerimientos que desea el cliente, aplicando transformaciones directas sobre datos y evitando la manipulación compleja de estados globales.

El objetivo es aplicar los conceptos fundamentales del paradigma funcional vistos en la materia, adaptándolos a una implementación práctica en Lisp, facilitando el tratamiento de información de manera más estructurada y legible.

A continuación, se describe la librería seleccionada y su función dentro del sistema implementado.

Local-time es una librería utilizada para la gestión de fechas, horas y zonas horarias en Common Lisp. Permitiendo trabajar con información temporal de forma más comprensible que los valores numéricos basados en el tiempo Unix (Epoch). Su incorporación en el proyecto permite representar y manipular datos temporales de manera clara, mejorando la legibilidad y precisión del sistema.

Bitácora de depuración

El **primer error** surgió durante las pruebas de la función `transicion`. Inicialmente utilizamos cadenas de texto (strings) en algunas pruebas y símbolos en otras. La función realizaba las comparaciones mediante `eq`, que funciona correctamente con símbolos, pero no con cadenas de caracteres.

```
[2]> (defun transicion (color-actual cambiar-a)
      (cond
        ((and (eq color-actual 'en-rojo) (eq cambiar-a
          (list color-actual "cambiar-a-verde")))
          ((and (eq color-actual 'en-verde) (eq cambiar-a
            (list color-actual "cambiar-a-amarillo")))
            ((and (eq color-actual 'en-amarillo) (eq cambia
              (list color-actual "cambiar-a-rojo")))
              (t (list color-actual 'accion-por-defecto)))
          TRANSICION
[3]> (transicion "en-rojo" 'verde)
("en-rojo" ACCION-POR-DEFECTO)
[4]>
```

Al corregir:

```
[3]> (transicion 'en-rojo 'verde)
(EN-ROJO "cambiar-a-verde")
```

El **segundo error** fue en la función `ciclos-por-tiempo`, el error conceptual fue que incorrectamente se usó la función `nth`, pasando una lista como primer parámetro y un número como segundo parámetro. El orden correcto sería (`nth` índice lista), clisp intentó interpretar la lista (rojo verde amarillo) como si fuera un índice numérico, generando el error de ejecución.

```
Break 4 [7]> (defun ciclos-por-tiempo (minutos)
  (nth-value 'rojo verde amarillo) (floor (* minutos 60) 216))
)

CICLOS-POR-TIEMPO
Break 4 [7]> (ciclos-por-tiempo 20)

*** - NTH: (ROJO VERDE AMARILLO) is not a non-negative integer
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
ABORT          :R2      Abort debug loop
ABORT          :R3      Abort debug loop
ABORT          :R4      Abort debug loop
ABORT          :R5      Abort debug loop
ABORT          :R6      Abort main loop
Break 5 [8]> |
```

Luego de revisar la teoría dada por la cátedra, identificamos la causa del problema y corregimos el orden de los parámetros.

```
Break 5 [8]> (defun ciclos-por-tiempo (minutos)
  (nth-value 0 (floor (* minutos 60) 216))
)

CICLOS-POR-TIEMPO
Break 5 [8]> (ciclos-por-tiempo 20)

5
Break 5 [8]> |
```

El **tercer error** que nos encontramos es en la extensión 2 de la iteración 2, en la función `escribir-registros`. Al intentar implementar la recursión para recorrer la lista de registros, cometimos el error de volver a llamar a la función utilizando la misma lista en lugar de avanzar al siguiente elemento mediante (`rest` datos). Nunca llegaba al caso base (`null` datos) y la función entraba en una recursión infinita, dando `stack overflow`.

```
(defun escribir-registros (datos stream)
  (cond
    ((null datos) nil)
    (t
     (format stream
              "~A - Transicion: ~A -> ~A~%"
              (local-time:format-timestring
               nil
               (local-time:now)
               :format '(:year "-" (:month 2) "-")
               (:day 2) " " (:hour 2) ":" (:min 2) ":" (:sec 2)))
              (second (first datos))
              (third (first datos)))
     (escribir-registros
      (rest datos)
      stream))))
```

```
[6]> (with-open-file (stream
  "informe-ejecucion-semaforo"
  :direction :output
  :if-exists :supersede)
  (escribir-registros '((100 rojo verde)) stream))

*** - Program stack overflow. RESET
```

La solución consistió en corregir la llamada recursiva para que procesara el resto de la lista en cada iteración:

(escribir-registros (rest datos) stream)

```
(escribir-registros
  (rest datos)
  stream)))
```

De esta manera, la lista se reducía progresivamente hasta llegar al caso base y finalizar correctamente la ejecución.

El **cuarto error** fue no incluir como asigna la consigna el formato de la librería, local-time y después olvidarnos de cargarlo en la consola.

```
Tiempo 100: la luz ha cambiado de ROJO a VERDE
Tiempo 220: la luz ha cambiado de VERDE a AMARILLO
Tiempo 226: la luz ha cambiado de AMARILLO a ROJO
```

```
(third (first datos)))
(escribir-registros (rest datos) stream)))
*** - READ en #<INPUT CONCATENATED-STREAM #<INPUT UNBUFFERED FILE-STREAM CHARACTER @35>>: no existe ningun paquete con el
nombre "LOCAL-TIME"
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
```

La solución fue cargar la librería antes de ejecutar el programa y lo verificamos con el log, donde en el informe, el horario y fecha salen correctamente.

```
Break 1 [8]> (ql:quickload :local-time)
To load "local-time":
  Load 1 ASDF system:
    local-time
; Loading "local-time"

(:LOCAL-TIME)
Break 1 [8]> (informe '((100 rojo verde) (220 verde amarillo)(226 amarillo rojo)))
```

```
=====
2026-06-15 18:16:10 - Transicion: ROJO -> VERDE
2026-06-15 18:16:10 - Transicion: VERDE -> AMARILLO
2026-06-15 18:16:10 - Transicion: AMARILLO -> ROJO

--- Fin del Informe ---|
```

Lenguaje asignado

Además, se nos encomendó investigar un lenguaje de programación, Clojure, para redefinir las funciones timer y transición.

Fue creado en 2007 por Rich Hickey. Haciendo énfasis en el paradigma funcional con el objetivo de reducir la complejidad asociada a la programación concurrente. Si en la programación concurrente dos funciones usan el mismo dato al mismo tiempo, puede ocurrir un error porque ambas intentan modificar ese valor a la vez y terminan pisándose o borrándose la información entre sí. Esto pasa porque una función puede leer un dato viejo antes de que la otra termine de actualizarlo, haciendo que el resultado final esté mal o el programa falle. La programación funcional tiene la característica que los datos son inmutables (no se pueden cambiar).

Clojure es un lenguaje de programación funcional de la familia Lisp que se ejecuta sobre la Máquina Virtual de Java (JVM). Está pensado desde cero para trabajar con datos estables y seguros, lo que permite armar sistemas grandes y robustos que no se rompen cuando hacen muchas tareas en paralelo.

Es utilizado principalmente en el desarrollo de software backend, sistemas distribuidos, análisis de datos y aplicaciones donde la concurrencia es un factor crítico. Su enfoque funcional y su integración con el ecosistema Java lo convierten en una opción atractiva para entornos empresariales. Entre las empresas que utilizan Clojure se encuentran Nubank, CircleCI, Walmart y Salesforce, principalmente en servicios backend y sistemas de procesamiento de alto rendimiento.

La principal ventaja al mapear los estados es que las palabras clave o keywords (como :rojo) funcionan como identificadores únicos que el lenguaje procesa internamente mediante una función de dispersión. A nivel de rendimiento, esto hace que las comparaciones de estados sean extremadamente rápidas. En lugar de tener que revisar la estructura del dato, letra por letra (comparación estructural), la computadora va directo a la dirección de memoria donde está guardada la referencia. Al reducir la operación a una simple lectura de dirección, el sistema consume muchísima menos memoria y procesa los cambios de forma casi instantánea, reducen el costo computacional y mejoran la legibilidad del código.

A su vez, Clojure prohíbe la mutabilidad directa de estructuras de datos, por lo que no es posible modificar variables de estado como en lenguajes imperativos. En su lugar, el manejo del tiempo en sistemas como un simulador semafórico se realiza mediante la creación de nuevos estados inmutables en cada transición. Esto significa que, en lugar de modificar una variable existente, el programa genera una nueva versión del estado del sistema en cada cambio de ciclo. De esta forma, el “tiempo” se modela como una secuencia de estados encadenados, donde cada estado depende del anterior. Este enfoque permite evitar efectos secundarios, facilita la concurrencia y hace que el comportamiento del sistema sea más predecible y fácil de razonar.

Conclusión:

En este trabajo se aplicaron los conceptos fundamentales del paradigma de programación funcional mediante una implementación en Common Lisp, utilizando funciones para transformar datos de entrada en resultados de manera clara y estructurada.

El uso de la librería local-time permitió mejorar el manejo de información temporal, aportando mayor legibilidad y precisión en el sistema. Además, el análisis del lenguaje Clojure ayudó a comprender el enfoque de inmutabilidad y su relación con el diseño funcional.

En conclusión, el trabajo permitió reforzar los conceptos teóricos vistos en la materia y aplicarlos en una solución práctica, destacando las ventajas del paradigma funcional en la organización y comprensión del código.

"If you want everything to be familiar, you'll never learn anything new". - Rich Hickey

Bibliografía:

FACENA – UNNE (2026). PARTE I: INTRODUCCION. TEMA 1: PARADIGMAS y Paradigmas de Programación

FACENA – UNNE (2026). Teoría Concurrencia y Paralelismo.

Microsoft Build 2026: <https://learn.microsoft.com/es-es/dotnet/fsharp/tutorials/functional-programming-concepts>

Clojure. Página oficial del lenguaje. <https://clojure.org/>

Clojure Guides. *Documentation and Learning Resources*. <https://clojure.org/guides/>

Compilador Clojure online: <https://www.mycompiler.io/es/new/clojure>